

WHERE Subquery

Used for complex filtering logic and makes query more flexible and dynamic.

COMPARISON OPERATORS

Used to filter data by comparing two values



Comparison Operators

=	Equal	WHERE Sales = (SELECT AVG(Sales) FROM ORDERS)
!= <>	Not Equal	WHERE Sales != (SELECT AVG(Sales) FROM ORDERS)
>	Greater than	WHERE Sales > (SELECT AVG(Sales) FROM ORDERS)
<	Less than	WHERE Sales < (SELECT AVG(Sales) FROM ORDERS)
>=	Greater than or equal to	WHERE Sales >= (SELECT AVG(Sales) FROM ORDERS)
<=	Less than or equal to	WHERE Sales <= (SELECT AVG(Sales) FROM ORDERS)



Subquery in WHERE Clause Comparison Operators

```
SELECT column1, column2,...  
  
FROM table1  
  
WHERE column = ( SELECT column FROM table2 WHERE condition )
```

Main Query

Subquery



Subquery in WHERE Clause Comparison Operators

```
SELECT column1, column2,...  
  
FROM table1  
  
WHERE column = ( SELECT column FROM table2 WHERE condition )
```

Main Query

Subquery

Rules

Only Scalar Subqueries are allowed to be used

SQLQuery2.sql - D:\8QBU\Youtube (70) - SQLQuery1.sql - D:\8QBU\Youtube (81)

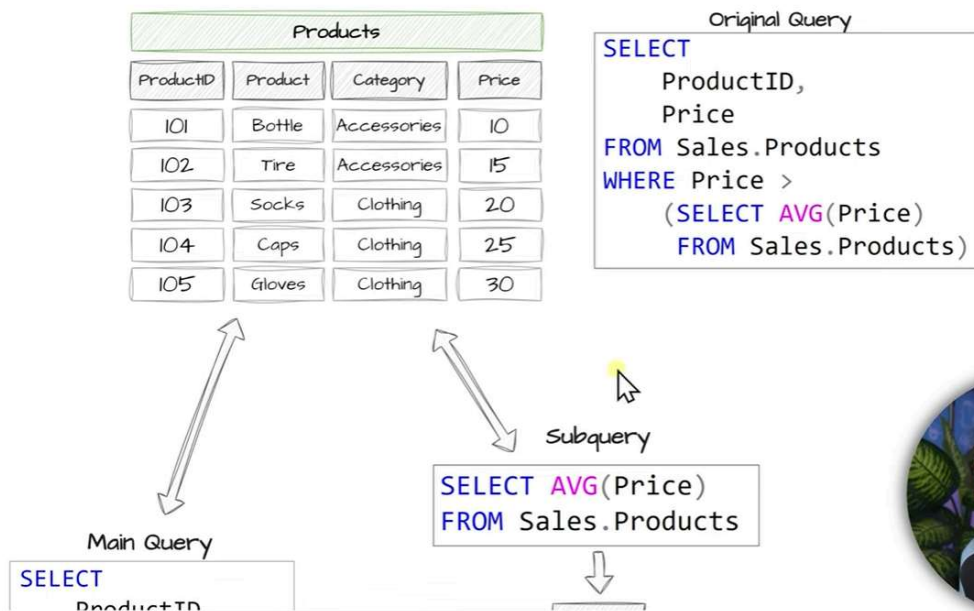
```
-- Find the products that have a price higher than the average price of all products

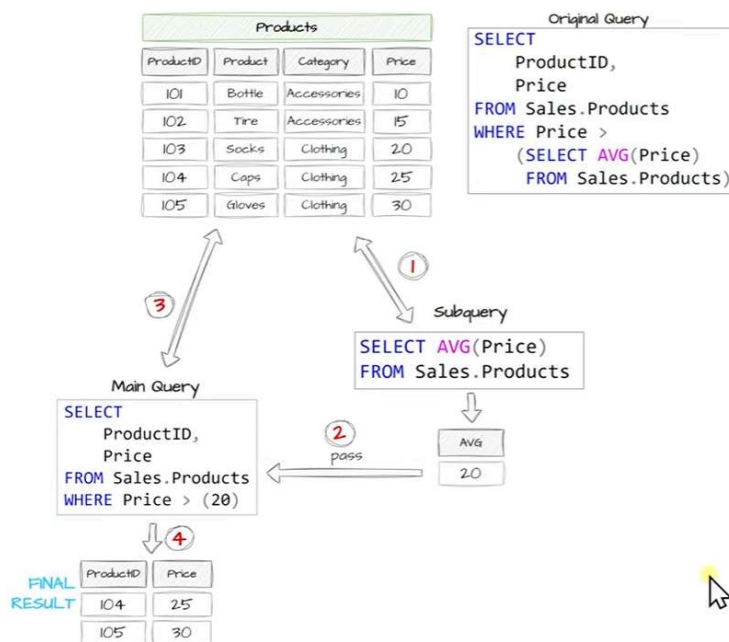
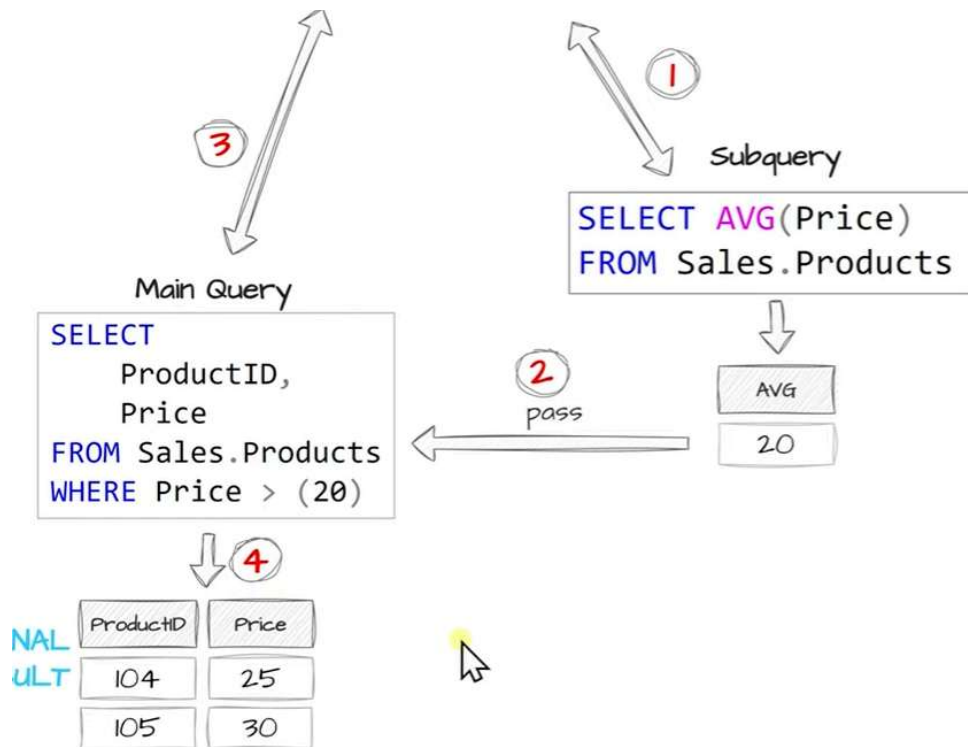
-- Main Query
SELECT
ProductID,
Price,
(SELECT AVG(Price) FROM Sales.Products) AvgPrice
FROM Sales.Products
WHERE Price > (SELECT AVG(Price) FROM Sales.Products)
```

183 %

Results Messages

	ProductID	Price	AvgPrice
1	104	25	20
2	105	30	20



FILTERING (SINGLE VALUE)

```
SELECT  
*  
FROM Sales.Orders  
WHERE CustomerID = 1
```

FILTERING (LIST OF VALUES)

```
SELECT  
*  
FROM Sales.Orders  
WHERE CustomerID IN (1,2,3,4)
```

IN OPERATOR

Checks whether a value matches any value from a list.



Subquery in
WHERE Clause
In Operator

```
SELECT column1, column2,...  
  
FROM table1  
  
WHERE column IN ( SELECT column FROM table2 WHERE condition )
```

Main Query

Subquery

SQLQuery5.sql - D:\8QBU\Youtube (60) | SQLQuery5.sql - D:\8QBU\Youtube (55)* | SQLQuery4.sql - D:\8QBU\Youtube (71)* | SQLQuery3.sql - D:\8QBU\Youtube (71)*

```
-- Show the details of orders made by customers in Germany
--Main Query
SELECT
*
FROM Sales.Orders
WHERE CustomerID IN
(
SELECT
CustomerID
FROM Sales.Customers
WHERE Country = 'Germany' )
```

201 %

	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Quantity
1	3	101	1	5	2025-01-10	2025-01-25	Delivered	8157 W. Book	8157 W. Book	2
2	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane		2
3	7	102	1	1	2025-02-15	2025-02-27	Delivered	136 Balboa Court		2
4	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4311 Clay Rd	3

Query executed successfully

DESKTOP-848BQBU\SQLEXPRESS | DESKTOP-848BQBU\Youtube



SQL TASK

Show the details of orders for customers
who are **not** from Germany.

SQLQuery7.sql - D:\8QBU\Youtube (59)* | SQLQuery6.sql - D:\8QBU\Youtube (60) | SQLQuery5.sql - D:\8QBU\Youtube (55)* | SQLQuery4.sql - D:\8QBU\Youtube (71)*

```
-- Show the details of orders made by customers in Germany
--Main Query
SELECT
*
FROM Sales.Orders
WHERE CustomerID IN
(
SELECT
CustomerID
FROM Sales.Customers
WHERE Country != 'Germany' )
```

201 %

	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Quantity
1	1	101	2	3	2025-01-01	2025-01-05	Delivered	9833 Mt. Dias Blv.	1226 Shoe St.	1
2	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	NULL	1
3	5	104	2	5	2025-02-01	2025-02-05	Delivered	NULL	NULL	1
4	6	104	3	5	2025-02-05	2025-02-10	Delivered	1792 Belmont Rd.	NULL	2
5	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way		2
6	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	NULL	0

Query executed successfully

DESKTOP-848BQBU\SQLEXPRESS | DESKTOP-848BQBU\Youtube



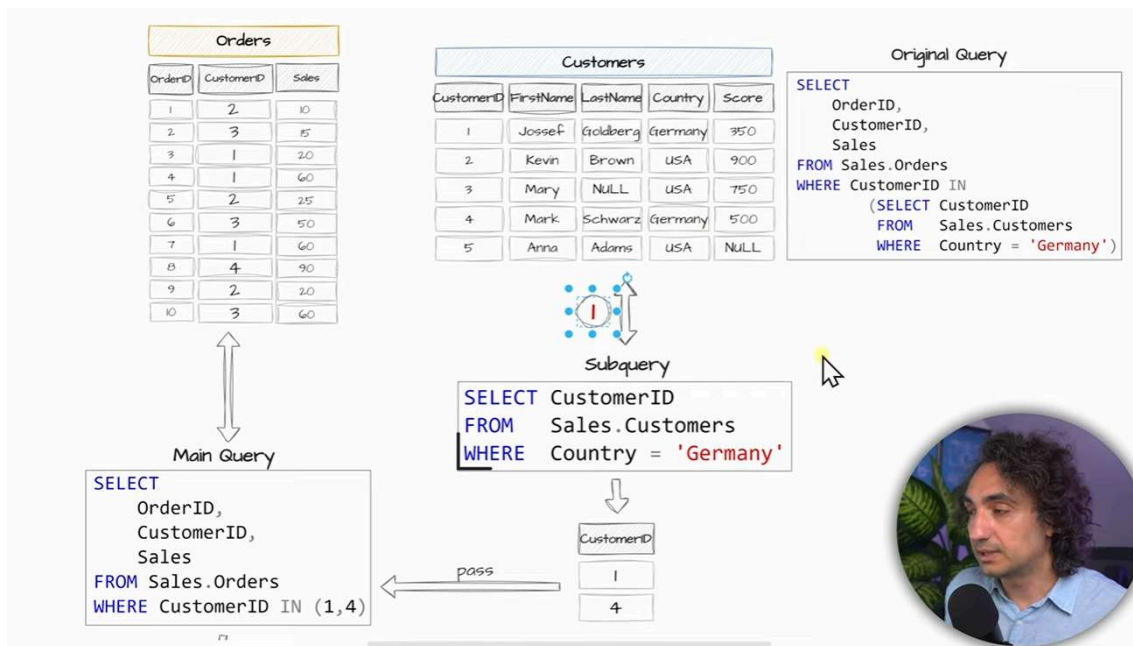
SQLQuery7.sql - D:\8QBU\Youtube (59) * x SQLQuery6.sql - D:\8QBU\Youtube (60) SQLQuery5.sql - D:\8QBU\Youtube (55) * SQLQuery4.sql - D:\8QBU\Youtube (7)

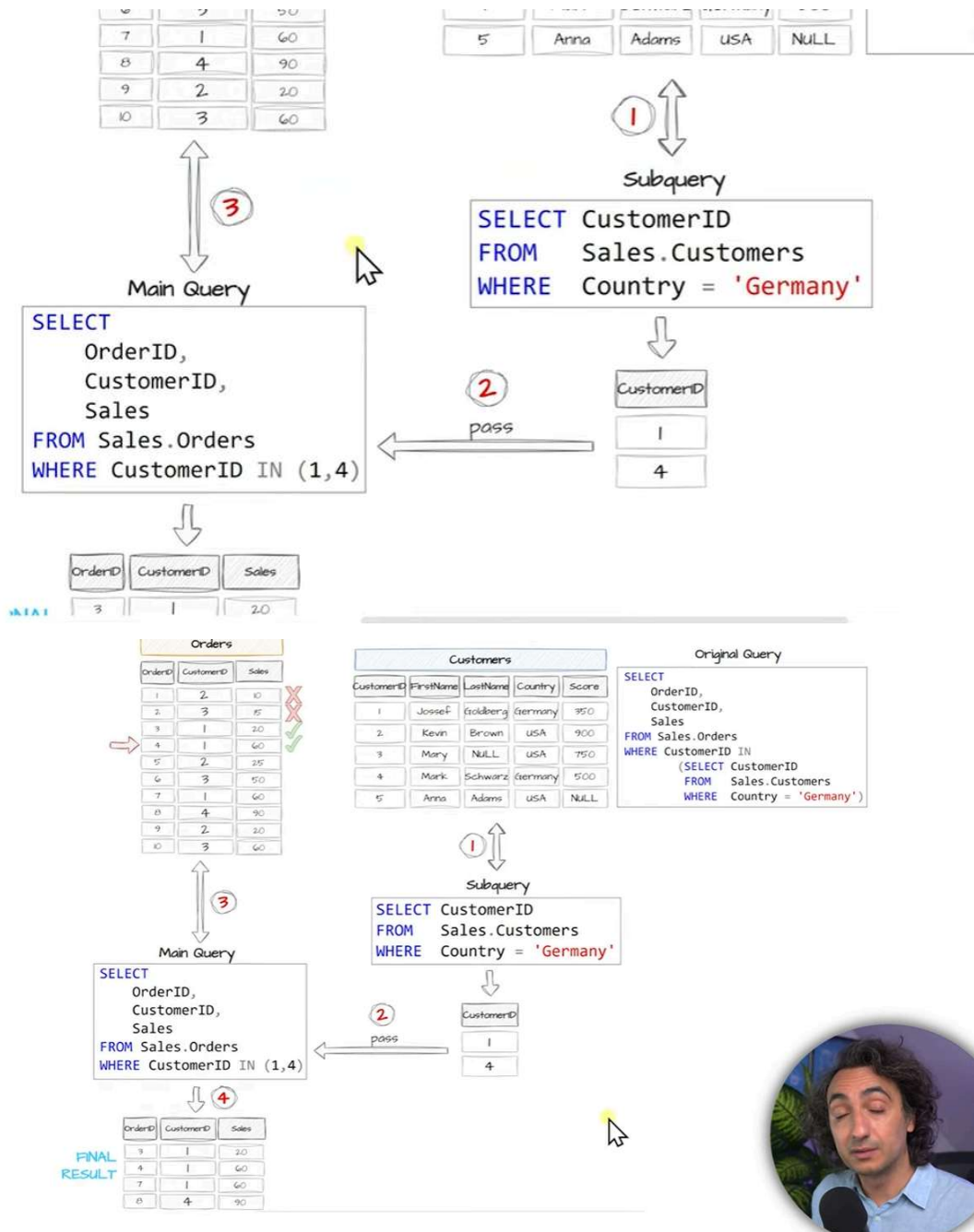
```
-- Show the details of orders made by customers in Germany
--Main Query
SELECT
*
FROM Sales.Orders
WHERE CustomerID NOT IN
(SELECT
CustomerID
FROM Sales.Customers
WHERE Country = 'Germany')
```

201 %

Results Messages

	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Qua
1	1	101	2	3	2025-01-01	2025-01-05	Delivered	9833 Mt. Dias Blv.	1226 Shoe St.	1
2	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	NULL	1
3	5	104	2	5	2025-02-01	2025-02-05	Delivered	NULL	NULL	1
4	6	104	3	5	2025-02-05	2025-02-10	Delivered	1792 Belmont Rd.	NULL	2
5	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way	NULL	2
6	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	NULL	0





Subquery

ANY | ALL

CASE WHEN Sales > 50

ANY OPERATOR

Checks if a value matches **ANY** value within a list.

Used to check if a value is true for **AT LEAST** one of the values in a list.



Subquery in
WHERE Clause
ALL Operator

```
SELECT column1, column2,...  
  
FROM table1  
  
WHERE column < ALL( SELECT column FROM table1 WHERE condition )
```

Main Query

Subquery

SQLQuery3.sql - D...8QBU\Youtube (57)* SQLQuery2.sql - D...8QBU\Youtube (56)* SQLQuery1.sql - D...8QBU\Youtube (52)*

```
-- Find female employees whose salaries are greater
-- than the salaries of any male employees

-- Main Query
SELECT
    EmployeeID,
    FirstName,
    Salary
FROM Sales.Employees
WHERE Gender = 'F'
AND Salary > ANY (SELECT Salary FROM Sales.Employees WHERE Gender = 'M');

SELECT FirstName, Salary FROM Sales.Employees WHERE Gender = 'M';
```

166 %

Results Messages

	EmployeeID	FirstName	Salary
1	3	Mary	75000

	FirstName	Salary
1	Frank	55000
2	Kevin	65000
3	Michael	90000

Query executed successfully.



SQLQuery3.sql - D...8QBU\Youtube (57)* SQLQuery2.sql - D...8QBU\Youtube (56)* SQLQuery1.sql - D...8QBU\Youtube (52)*

```
-- Find female employees whose salaries are greater
-- than the salaries of any male employees

-- Main Query
SELECT
    EmployeeID,
    FirstName,
    Salary
FROM Sales.Employees
WHERE Gender = 'F'
AND Salary > ANY (SELECT Salary FROM Sales.Employees WHERE Gender = 'M');

SELECT FirstName, Salary FROM Sales.Employees WHERE Gender = 'M';
```

166 %

Results Messages

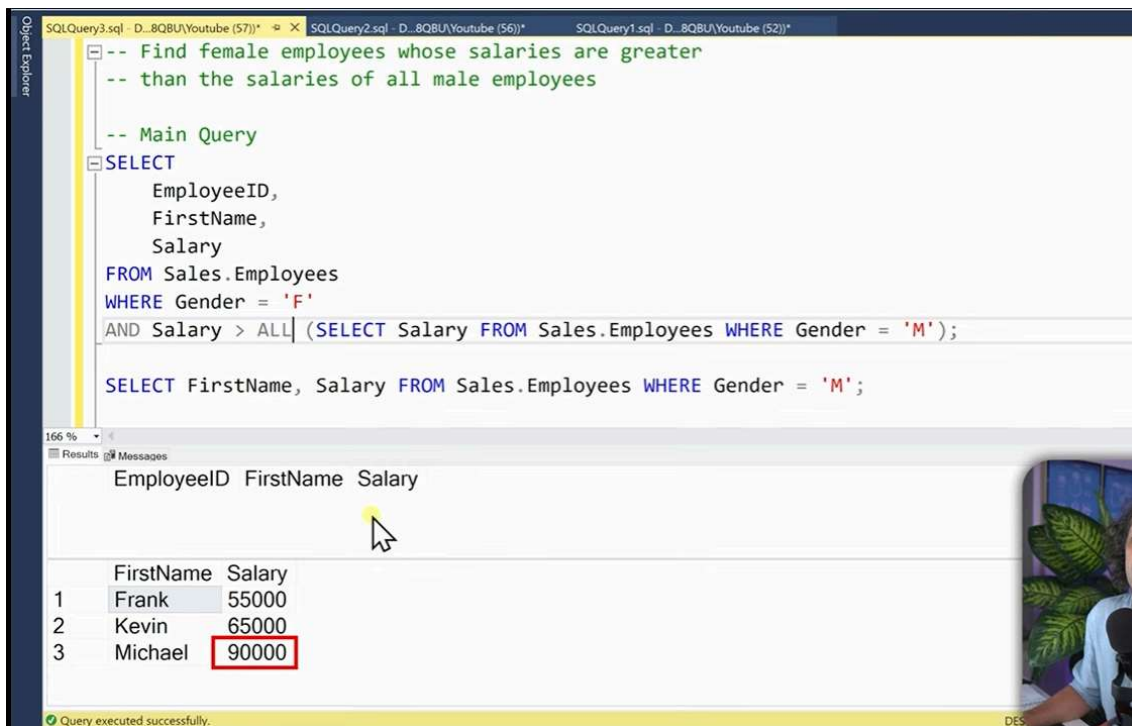
	EmployeeID	FirstName	Salary
1	3	Mary	75000
2	5	Carol	55000

ALL OPERATOR

Checks if a value matches **ALL** values within a list.

SQL TASK

Find female employees whose salaries are greater than the salaries of all male employees



The screenshot shows the SQL Server Enterprise Manager interface. The query editor displays a SQL query designed to find female employees whose salaries are greater than those of all male employees. The query uses the **ALL** operator. Below the query, the Results pane shows the output of the query, which includes a list of female employees and their salaries. The results are as follows:

EmployeeID	FirstName	Salary
1	Frank	55000
2	Kevin	65000
3	Michael	90000

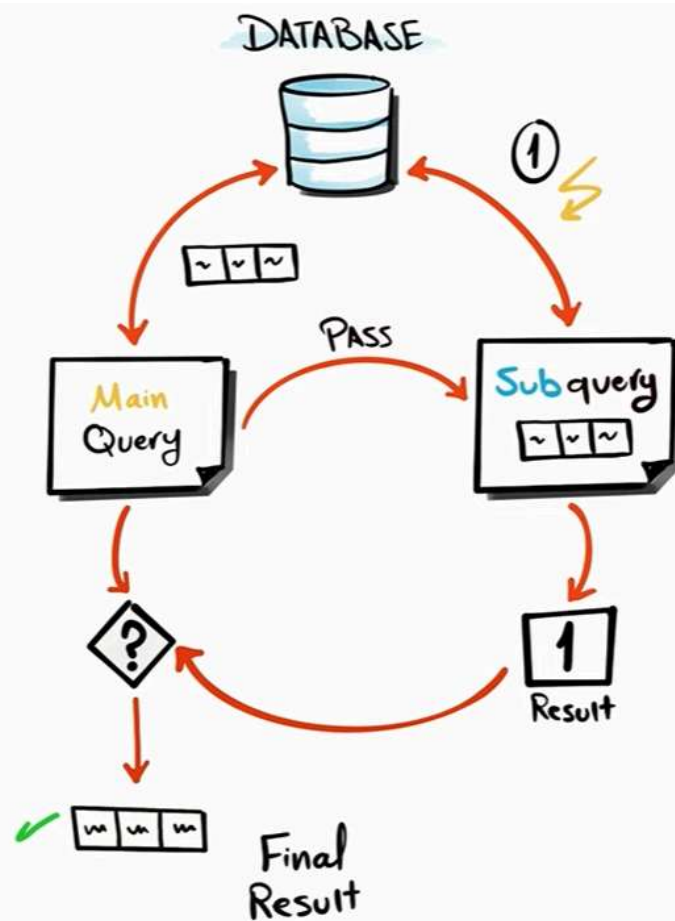
The salary value 90000 for Michael is highlighted with a red box. A status bar at the bottom indicates "Query executed successfully."

NON-CORRELATED SUBQUERY

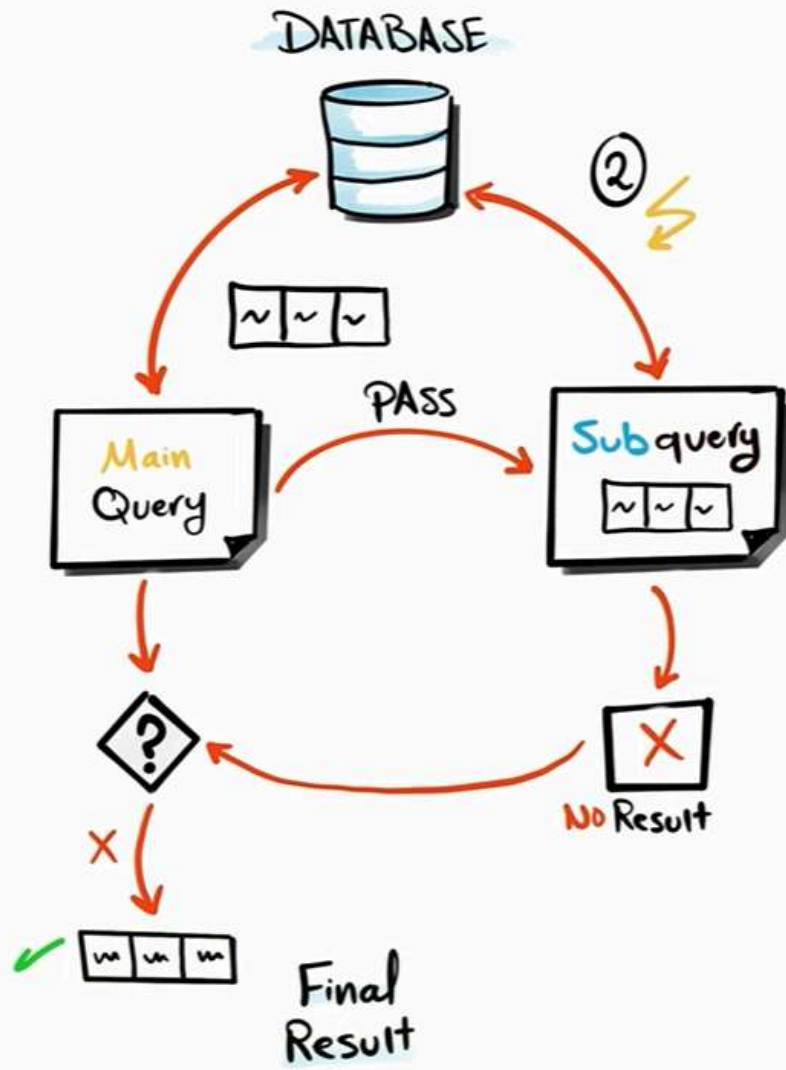
A Subquery that can run independently from the Main Query

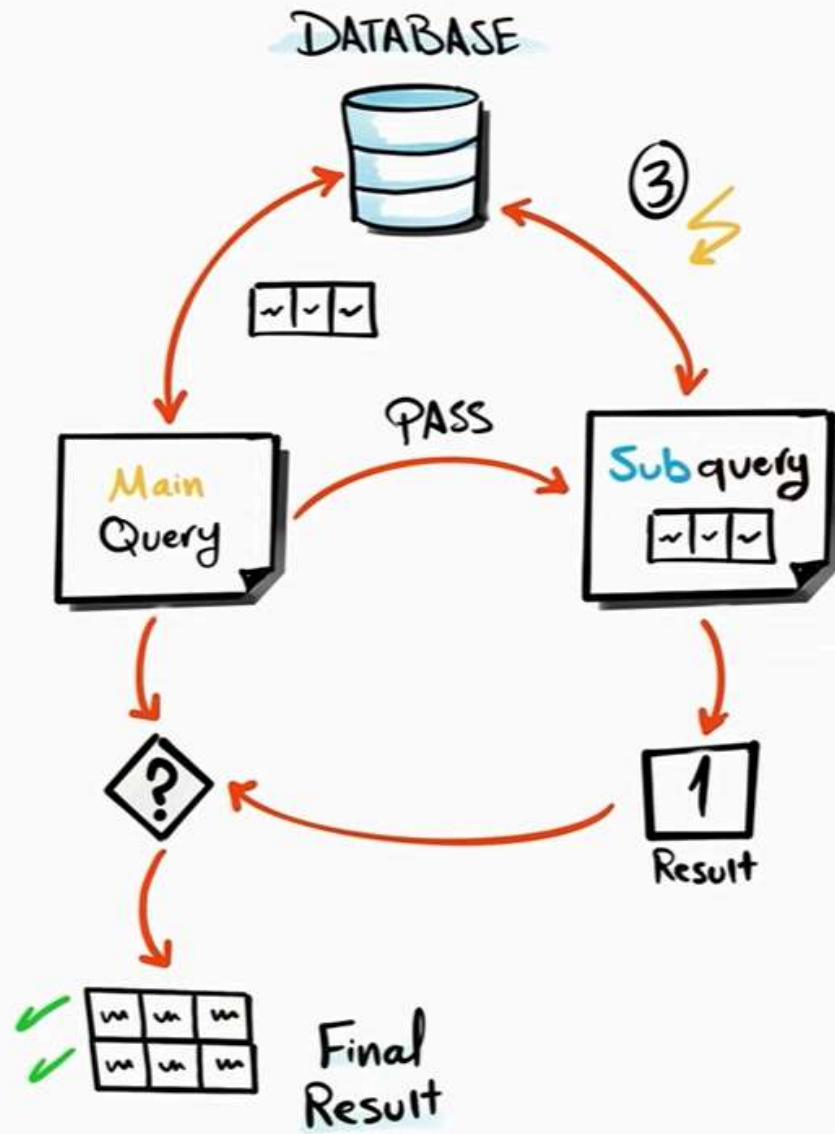
CORRELATED SUBQUERY

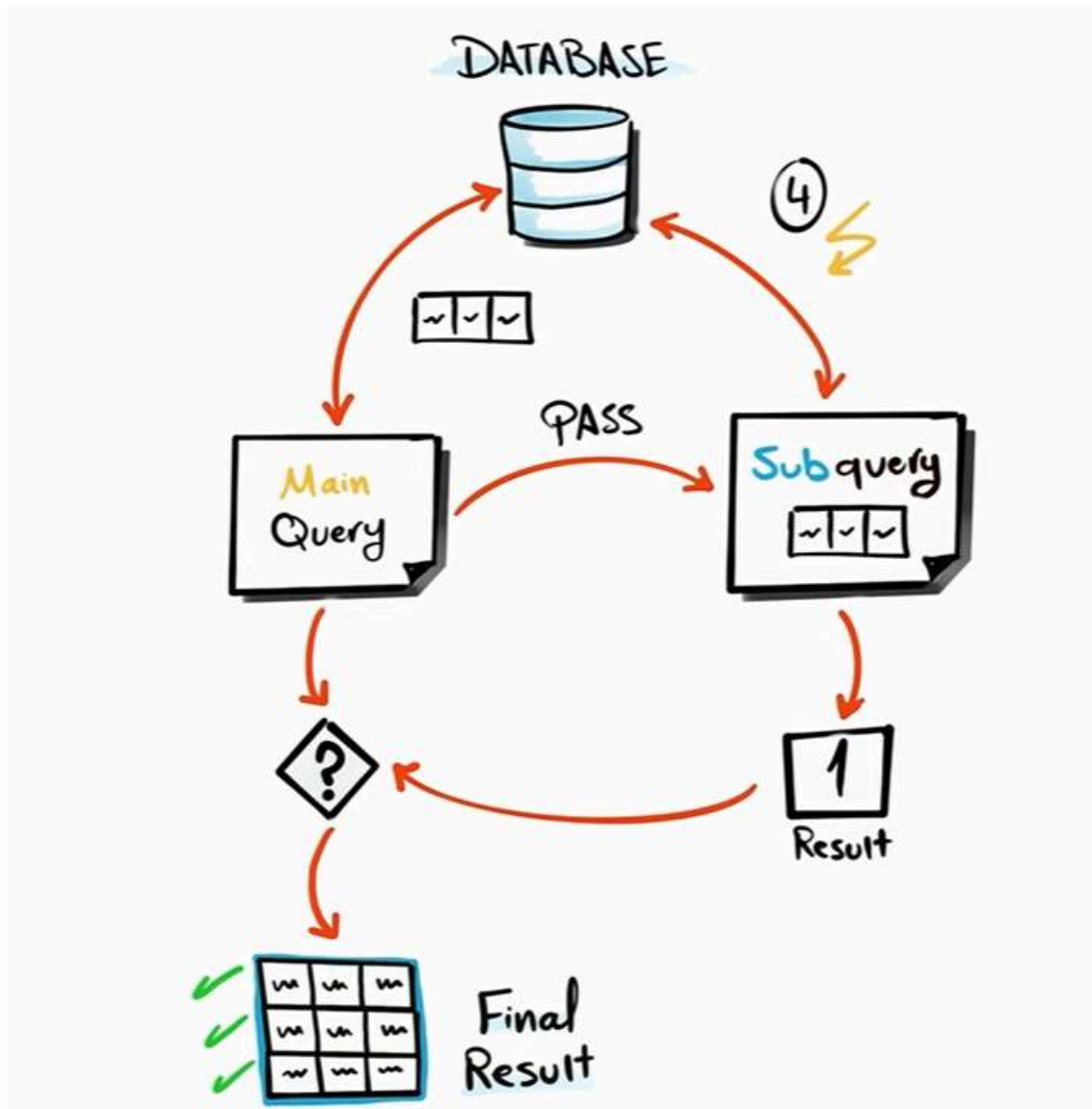
A Subquery that relays on values from the Main Query



This process works row by row.

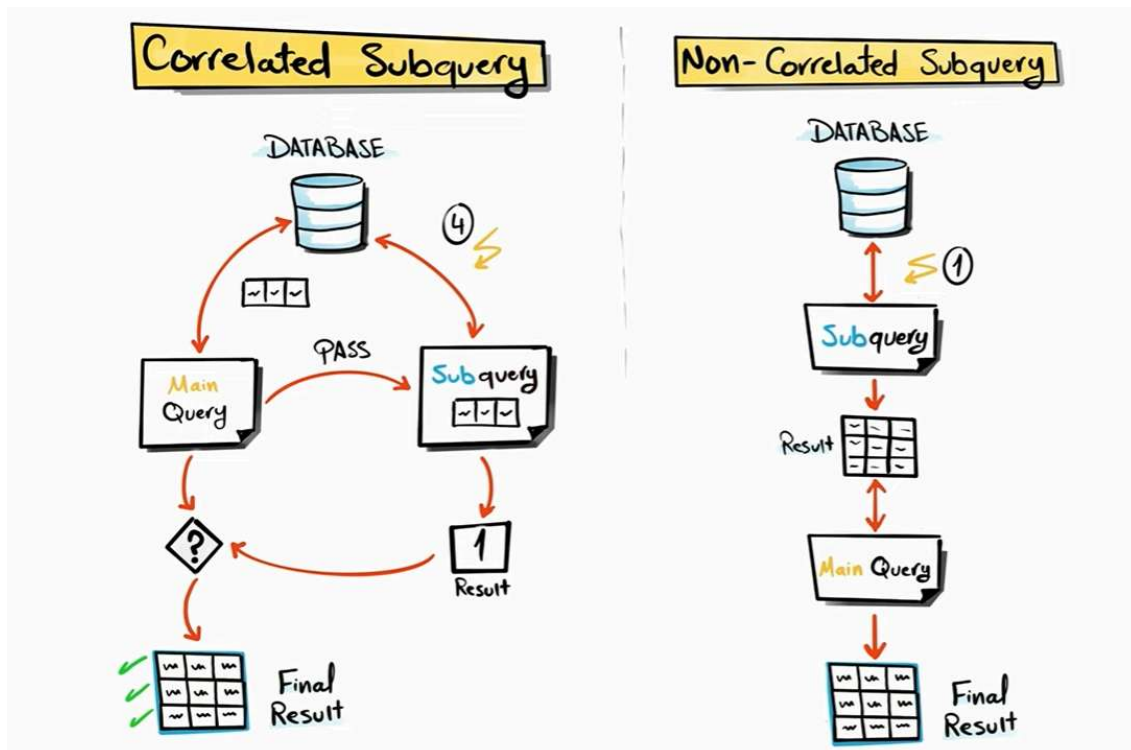






Corrorelated subquery Depending on main query. Subquery executed for each row that's we gona get from main query so as an example we have four rows in main query and subquery executed four times for each row.

Iteration of each row happended that gona be retrieved from the main query for further expansion.



NO iterations in non - correlated subquery.
Query executed only once and not depend on the main query.

```

-- Show all customer details and find the total orders of each customer

-- Main Query
SELECT
*,
(SELECT COUNT(*) FROM Sales.Orders) TotalSales
FROM Sales.Customers

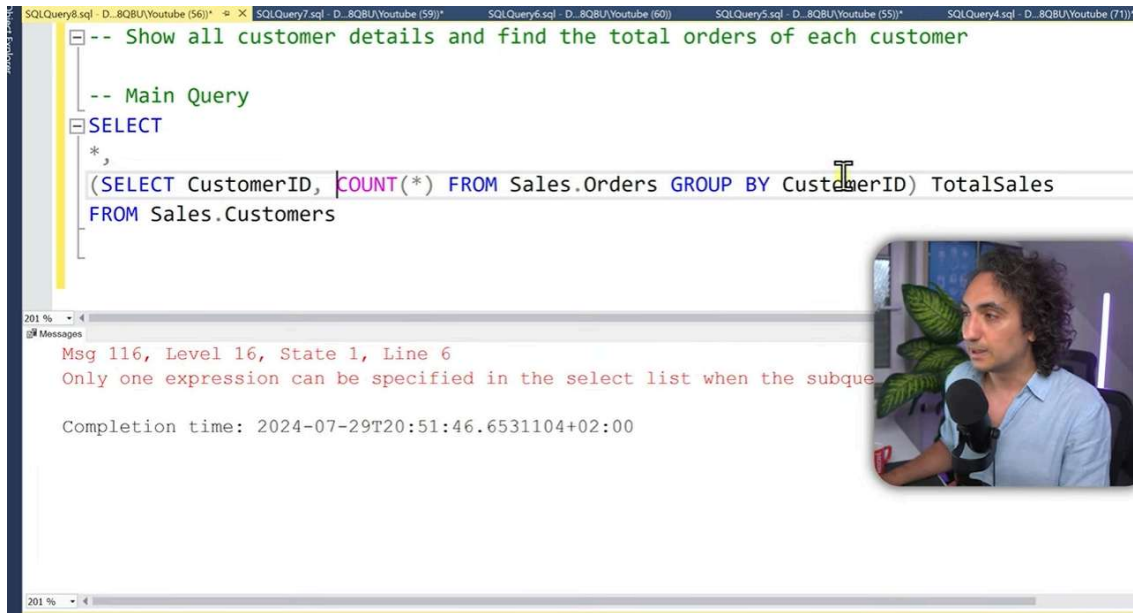
```

	CustomerID	FirstName	LastName	Country	Score	TotalSales
1	1	Jossef	Goldberg	Germany	350	10
2	2	Kevin	Brown	USA	900	10
3	3	Mary	NULL	USA	750	10
4	4	Mark	Schwarz	Germany	500	10
5	5	Anna	Adams	USA	NULL	10

Query executed successfully.
DESKTOP-84B8QB\SQLEXPRESS ... DESKTOP-84

Each customer will have different total order

We can not use group by subquery in select clause because it will return multiple records and we can use only scalar subquery in a select clause.



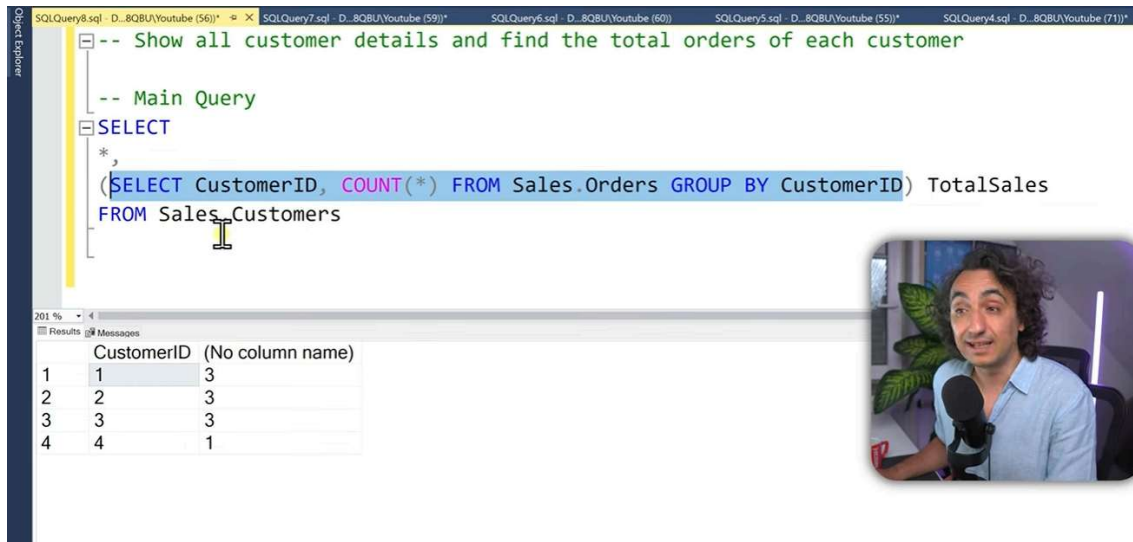
The screenshot shows a SQL query in SQL Server Enterprise Manager. The query is:

```
-- Show all customer details and find the total orders of each customer
-- Main Query
SELECT
*,
(SELECT CustomerID, COUNT(*) FROM Sales.Orders GROUP BY CustomerID) TotalSales
FROM Sales.Customers
```

The query is executed, and an error message is displayed:

Msg 116, Level 16, State 1, Line 6
Only one expression can be specified in the select list when the subquery is used.

Completion time: 2024-07-29T20:51:46.6531104+02:00



The screenshot shows the same SQL query as above, but with the results displayed in the Results pane:

```
-- Show all customer details and find the total orders of each customer
-- Main Query
SELECT
*,
(SELECT CustomerID, COUNT(*) FROM Sales.Orders GROUP BY CustomerID) TotalSales
FROM Sales.Customers
```

The Results pane shows a table with 4 rows and 3 columns:

	CustomerID	(No column name)
1	1	3
2	2	3
3	3	3
4	4	1



We can use now correlated subquery
Current query is totally non-correlated query.
Now we told query execute for specific customers not for the table.

SQLQuery8.sql - D:\80BU\Youtube (56)* x SQLQuery7.sql - D:\80BU\Youtube (59)* SQLQuery6.sql - D:\80BU\Youtube (60) SQLQuery5.sql - D:\80BU\Youtube (55)* SQLQuery4.sql - D:\80BU\Youtube (71)*

```
-- Show all customer details and find the total orders of each customer

-- Main Query
SELECT
*,
(SELECT COUNT(*) FROM Sales.Orders o WHERE o.CustomerID = c.CustomerID) TotalSales
FROM Sales.Customers c
```

201 %

	CustomerID	FirstName	LastName	Country	Score	TotalSales
1	1	Jossef	Goldberg	Germany	350	3
2	2	Kevin	Brown	USA	900	3
3	3	Mary	NULL	USA	750	3
4	4	Mark	Schwarz	Germany	500	1
5	5	Anna	Adams	USA	NULL	0



For each row subquery will be executed

```
-- Show all customer details and find the total orders of each customer

-- Main Query
SELECT
*,
(SELECT COUNT(*) FROM Sales.Orders o WHERE o.CustomerID = 1) TotalSales
FROM Sales.Customers c
```

201 %

	(No column name)
1	3



	Non-Correlated Subquery	Correlated Subquery
Definition	Subquery is independent of the main query	Subquery is dependent of the main query
Execution	Executed once and its result is used by the main query Can be executed on its Own	Executed for each row processed by the main query Can't be executed on its Own.
Easy to use	Easier to read	Harder to read and more complex
Performance	Executed only once leads to better Performance	Executed multiple times leads to bad Performance
Usage	Static Comparisons, Filtering with Constants	Row-by-Row Comparisons, Dynamic Filtering

Correlated Subquery

EXISTS

EXISTS

Check if a subquery returns any rows



Correlated Subquery in WHERE Clause EXISTS Operator

Main Query

```
SELECT column1, column2,...  
  
FROM Table2  
  
WHERE EXISTS ( SELECT 1  
                FROM Table1  
                WHERE Table1.ID = Table2.ID )
```

Subquery



How EXISTS Works?

For each row in
Main Query



Run Subquery

No Result?

Row of Main Query
is **excluded**

returns **Value** ?

Row of Main Query
is **included**

-- Show the details of orders made by customers in Germany

--Main Query

SELECT

*

FROM Sales.Orders o

WHERE EXISTS (SELECT 1

FROM Sales.Customers c

WHERE Country = 'Germany'

AND o.CustomerID = c.CustomerID)

Results

	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Qua
1	3	101	1	5	2025-01-10	2025-01-25	Delivered	8157 W. Book	8157 W. Book	2
2	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane		2
3	7	102	1	1	2025-02-15	2025-02-27	Delivered	136 Balboa Court		2
4	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4311 Clay Rd	3



-- Show the details of orders made by customers in Germany

--Main Query

SELECT

*

FROM Sales.Orders o

WHERE EXISTS (SELECT 3

FROM Sales.Customers c

WHERE Country = 'Germany'

AND o.CustomerID = c.CustomerID)

Results

	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Qua
1	3	101	1	5	2025-01-10	2025-01-25	Delivered	8157 W. Book	8157 W. Book	2
2	4	105	1	3	2025-01-20	2025-01-25	Shipped	5724 Victory Lane		2
3	7	102	1	1	2025-02-15	2025-02-27	Delivered	136 Balboa Court		2
4	8	101	4	3	2025-02-18	2025-02-27	Shipped	2947 Vine Lane	4311 Clay Rd	3



-- Show the details of orders made by customers in Germany

--Main Query

```
SELECT
*
FROM Sales.Orders o
WHERE NOT EXISTS (SELECT 1
                  FROM Sales.Customers c
                  WHERE Country = 'Germany'
                  AND o.CustomerID = c.CustomerID)
```

	OrderID	ProductID	CustomerID	SalesPersonID	OrderDate	ShipDate	OrderStatus	ShipAddress	BillAddress	Quantity
1	1	101	2	3	2025-01-01	2025-01-05	Delivered	9833 Mt. Dias Blv.	1226 Shoe St.	1
2	2	102	3	3	2025-01-05	2025-01-10	Shipped	250 Race Court	NULL	1
3	5	104	2	5	2025-02-01	2025-02-05	Delivered	NULL	NULL	1
4	6	104	3	5	2025-02-05	2025-02-10	Delivered	1792 Belmont Rd.	NULL	2
5	9	101	2	3	2025-03-10	2025-03-15	Shipped	3768 Door Way	NULL	2
6	10	102	3	5	2025-03-15	2025-03-20	Shipped	NULL	NULL	0

-- Show the details of orders made by customers not in Germany

--Main Query

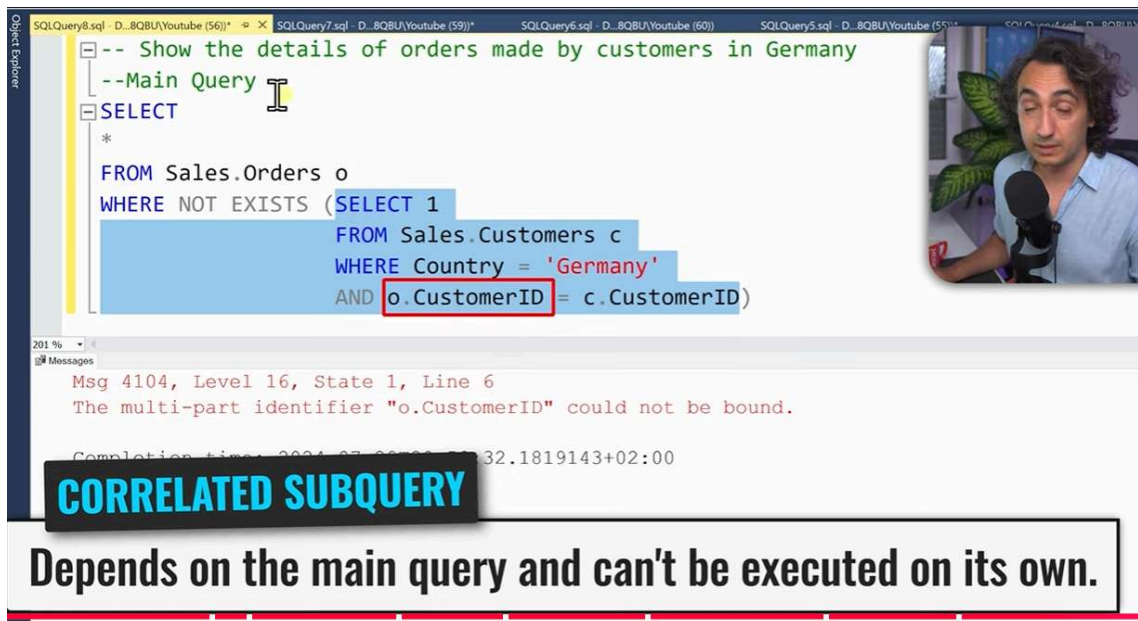
```
SELECT
*
FROM Sales.Orders
WHERE CustomerID NOT IN
      (SELECT
       CustomerID
       FROM Sales.Customers
       WHERE Country = 'Germany')
```

	CustomerID
1	1
2	4

NON-CORRELATED SUBQUERY

~~Independent of the main query and can be executed on its own~~

1:15:05 / 1:21:27 Exists Operator Subquery > DESKTOP-84B8QBU\SQLEXPRESS ... DESKTOP



The screenshot shows a SQL query in SQL Server Enterprise Manager. The query is a correlated subquery that attempts to select orders from the Sales.Orders table where there does not exist a customer in the Sales.Customers table from Germany with the same CustomerID. The subquery is highlighted in blue, and the correlation 'o.CustomerID' is highlighted in red. The error message at the bottom states: 'Msg 4104, Level 16, State 1, Line 6 The multi-part identifier "o.CustomerID" could not be bound.' A black box with the text 'CORRELATED SUBQUERY' is overlaid on the error message. A video inset in the top right corner shows a man speaking into a microphone.

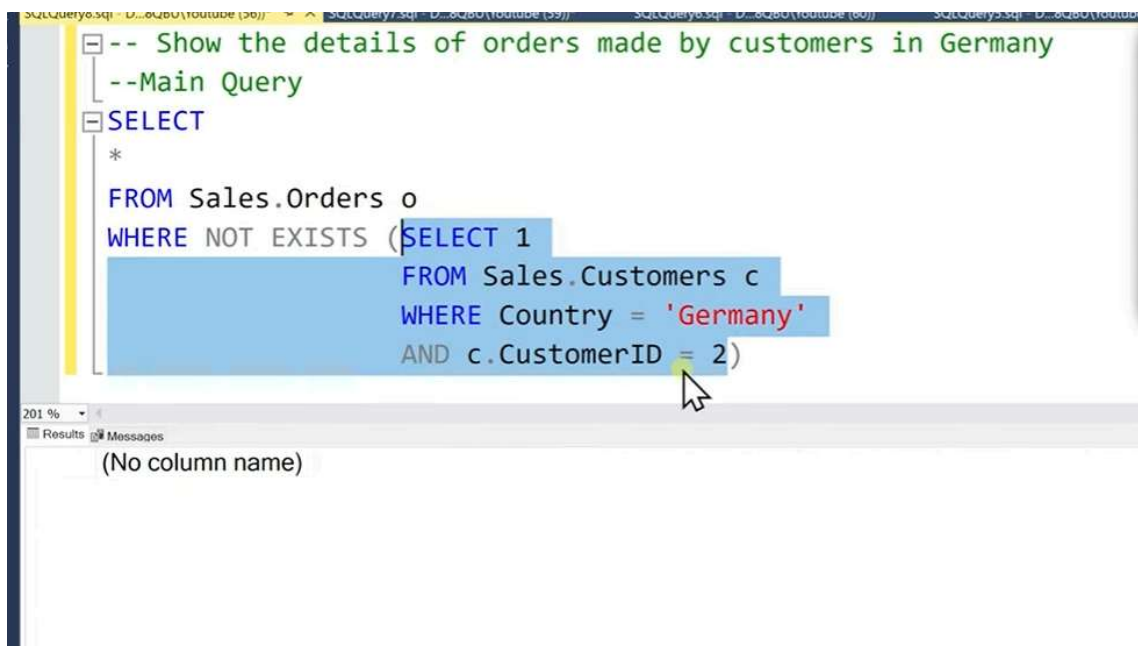
```
-- Show the details of orders made by customers in Germany
--Main Query
SELECT
*
FROM Sales.Orders o
WHERE NOT EXISTS (SELECT 1
FROM Sales.Customers c
WHERE Country = 'Germany'
AND o.CustomerID = c.CustomerID)
```

Msg 4104, Level 16, State 1, Line 6
The multi-part identifier "o.CustomerID" could not be bound.

Completion time: 2024-07-28 10:32:18.1819143+02:00

CORRELATED SUBQUERY

Depends on the main query and can't be executed on its own.

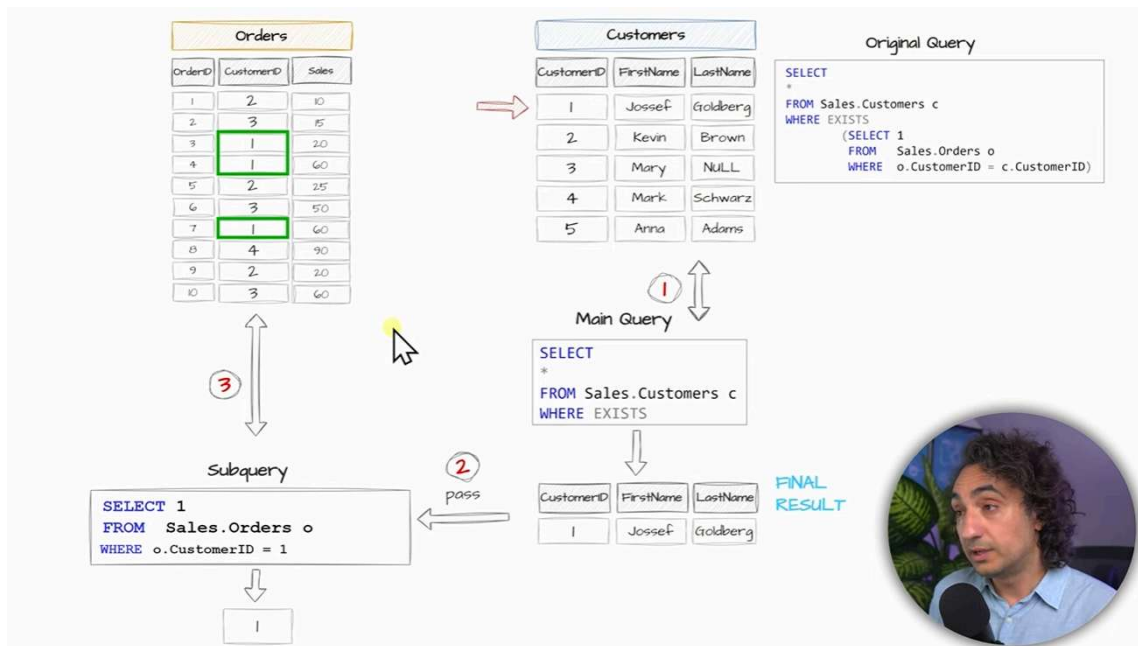
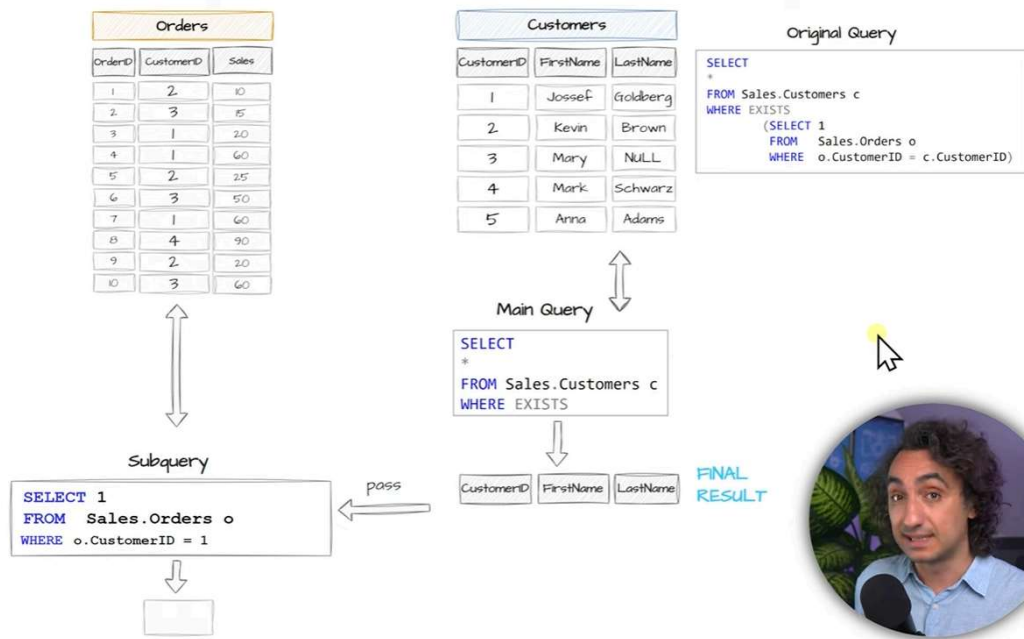


The screenshot shows the same SQL query as the previous one, but with a different error message. The subquery is highlighted in blue, and the correlation 'c.CustomerID' is highlighted in yellow. The error message at the bottom states: '(No column name)'. A mouse cursor is pointing at the end of the subquery.

```
-- Show the details of orders made by customers in Germany
--Main Query
SELECT
*
FROM Sales.Orders o
WHERE NOT EXISTS (SELECT 1
FROM Sales.Customers c
WHERE Country = 'Germany'
AND c.CustomerID = 2)
```

(No column name)

Correlated query is hard to understand and we can not run independently their subquery to test result.



Orders		
OrderID	CustomerID	Sales
1	2	10
2	3	15
3	1	20
4	1	60
5	2	25
6	3	50
7	1	60
8	4	90
9	2	20
10	3	60

Customers		
CustomerID	FirstName	LastName
1	Jossef	Goldberg
2	Kevin	Brown
3	Mary	NULL
4	Mark	Schwarz
5	Anna	Adams

Original Query

```
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
(
  SELECT 1
  FROM Sales.Orders o
  WHERE o.CustomerID = c.CustomerID
)
```



Main Query

```
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
```

CustomerID	FirstName	LastName
1	Jossef	Goldberg
2	Kevin	Brown
3	Mary	NULL
4	Mark	Schwarz
5	Anna	Adams

FINAL RESULT



3

Subquery

```
SELECT 1
FROM Sales.Orders o
WHERE o.CustomerID = 5
```



Orders		
OrderID	CustomerID	Sales
1	2	10
2	3	15
3	1	20
4	1	60
5	2	25
6	3	50
7	1	60
8	4	90
9	2	20
10	3	60

Customers		
CustomerID	FirstName	LastName
1	Jossef	Goldberg
2	Kevin	Brown
3	Mary	NULL
4	Mark	Schwarz
5	Anna	Adams

Original Query

```
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
(
  SELECT 1
  FROM Sales.Orders o
  WHERE o.CustomerID = c.CustomerID
)
```



Main Query

```
SELECT
*
FROM Sales.Customers c
WHERE EXISTS
```

CustomerID	FirstName	LastName
1	Jossef	Goldberg
2	Kevin	Brown
3	Mary	NULL
4	Mark	Schwarz

FINAL RESULT



3

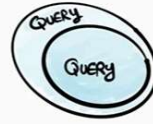
Subquery

```
SELECT 1
FROM Sales.Orders o
WHERE o.CustomerID = 5
```



SUBQUERY

Query inside Another Query



Breaks complex query into smaller, manageable pieces.

USE CASES

- Create Temporary Result Set.
- Prepare Data Before Joining Tables.
- Dynamic & Complex Filtering.
- Check the Existence of Rows From another Table. (EXISTS)
- Row By Row Comparison - Correlated Subquery -

